

MANJUSHREE_Week03_GATE_3 REPORT

Project Title

End-to-End Test Automation Framework using Selenium, Cucumber BDD, JUnit, and Maven.

Objective

The objective of this project is to develop a maintainable and scalable test automation framework for validating the functionality of a web-based e-commerce application. The framework follows the Behaviour Driven Development (BDD) approach to improve collaboration between testers, developers, and business stakeholders.

Technologies Used

- Java
- Selenium WebDriver
- Cucumber BDD
- JUnit 5
- Maven
- Page Object Model (POM)
- Allure Reports
- Extent Reports

Framework Architecture

The framework is designed using the Page Object Model pattern. The project structure consists of:

1. Feature Files
 - Contains business scenarios written in Gherkin language.
 - Uses Given, When, Then steps.
2. Step Definitions
 - Maps Gherkin steps to Java methods.
 - Contains automation logic for each scenario.

3. Page Classes

- Encapsulates web elements and actions.
- Improves code reusability and maintainability.

4. Base Classes

- Handles WebDriver initialization.
- Contains common methods and utilities.

5. Hooks

- Executes setup and teardown methods before and after scenarios.

6. Test Runner

- Executes feature files using JUnit and Cucumber.

Features Automated

Product Search

- Search for available products.
- Validate search results.
- Search for unavailable products.
- Verify "Showing 0 products" message.

Product Sorting

- Sort products by:
 - Price: Low to High
 - Price: High to Low
 - Validate sorting order.

Cart Functionality

- Add products to cart.
- Verify cart badge updates.
- Remove products from cart.
- Validate cart count.

Checkout Process

- Add product to cart.

- Proceed to checkout.
- Fill customer details.
- Place order successfully.

Sample BDD Scenario

Feature: Search Product

Scenario: Search an unavailable product

Given I open the catalog page

When I search for "InvalidProduct"

Then it should show "Showing 0 products"

Reporting

Allure Report

- Generates detailed execution reports.
- Displays passed, failed, and skipped test cases.
- Includes screenshots for failed scenarios.

Extent Report

- Provides graphical representation of execution results.
- Displays execution statistics and logs.

Advantages of the Framework

- Easy to maintain using Page Object Model.
- Human-readable test scenarios using Gherkin.
- Supports reusable components.
- Detailed reporting using Allure and Extent Reports.
- Supports tagging such as:
 - @smoke
 - @regression
 - @negative

Conclusion

The Selenium BDD Automation Framework provides an efficient and scalable solution for automating end-to-end web application testing. By combining Selenium with Cucumber BDD and Page Object Model, the framework achieves better readability, maintainability, and collaboration among team members while ensuring high-quality software delivery.

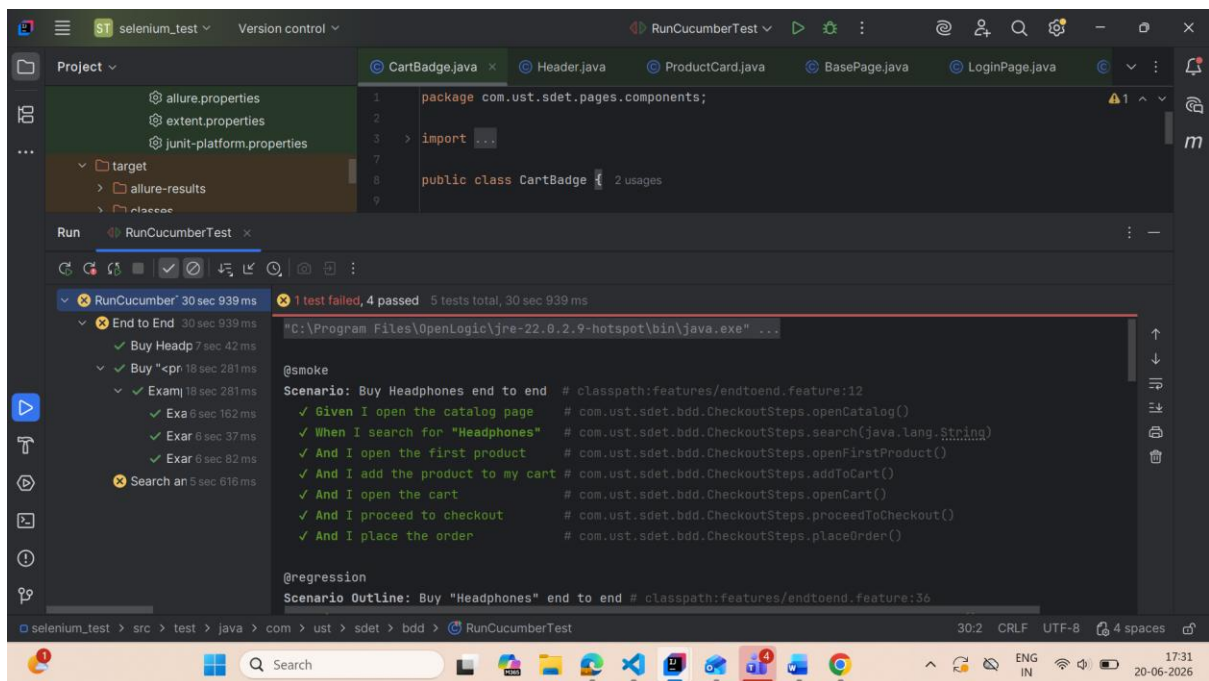
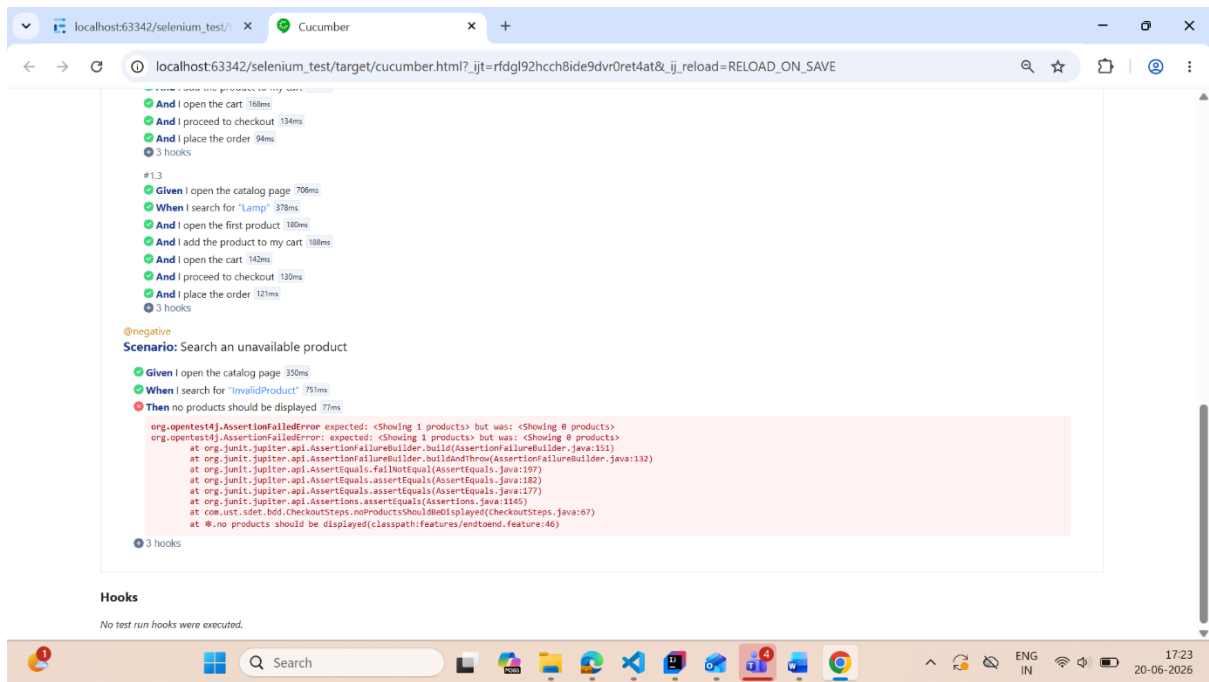
Evidence

This screenshot displays the Selenium Spark Report interface in a web browser. The URL is localhost:63342/selenium_test/test-output/SparkReport/Spark.html?_ijt=7g0jq842cllpkpitqa9tc8p7a9&_ij_reload=RELOAD_ON_SAVE. The report shows a test titled "End to End Checkout Flow" with a status of "Fail". The test description is: "As a shopper I want to search the catalog, add a product to my cart and complete checkout So that I can buy what I came for and the team can prove that journey works on every build". The test results are as follows:

Test Case	Status
Buy Headphones end to end	Pass
Buy "" end to end	Pass
Buy "Headphones" end to end	Pass
Buy "Shoes" end to end	Pass
Buy "Lamp" end to end	Pass

This screenshot displays the Cucumber HTML report interface in a web browser. The URL is localhost:63342/selenium_test/target/cucumber.html?_ijt=rfdgl92hcch8ide9dvr0ret4at&_ij_reload=RELOAD_ON_SAVE. The report shows a test titled "End to End Checkout Flow" with a status of "80% (4 / 5) passed". The test description is: "As a shopper I want to search the catalog, add a product to my cart and complete checkout So that I can buy what I came for and the team can prove that journey works on every build". The test results are as follows:

Test Case	Status
Buy Headphones end to end	Pass
Buy "" end to end	Pass
Buy "Headphones" end to end	Pass
Buy "Shoes" end to end	Pass
Buy "Lamp" end to end	Pass



Commands to Run

@Smoke - `mvn test -Dtest=RunCucumberTest "-Dcucumber.filter.tags=@smoke"`

@Regression - `mvn test -Dtest=RunCucumberTest "-Dcucumber.filter.tags=@regression"`

@Negative - `mvn test -Dtest=RunCucumberTest "-Dcucumber.filter.tags=@negative"`

